Prediction Markets on the Cost-Accounted Rho Calculus

A gap analysis, an oracle discipline, a market-launch configuration tuple, and an implementation blueprint for a Kalshi/Polymarket-class venue on F1R3Fi

L. G. Meredith
F1R3FLY.io lgreg.meredith@f1r3fly.io

June 2026

Abstract

The Peyton Jones–Eber–Seward combinators, realised in the cost-accounted rho calculus, give an executable, resource-safe settlement language for financial *instruments*. A prediction market, however, is a *venue*, and a venue is mostly the three things the combinator language deliberately does not model: a fungible bearer-position layer, a matching/automated-market-maker microstructure, and a resolution oracle robust against an adversarial, informally-specified world. This note closes that distance. We inventory the gaps between the existing F1R3Fi realisation and a venue with feature parity to Kalshi and Polymarket; we show that the position layer falls out of located purses and the acceptance gate almost for free (the Gnosis conditional-token conservation law *is* a linear funding proof); we give an oracle discipline in which a trusted computation lives behind a channel, non-repeatable oracle calls are secured not by replicated determinism—unattainable for, e.g., a high-temperature language model—but by *validator attribution*, a blame capability that Casper-style accountable safety already supplies, with the oracle's declared *specification* furnishing the falsification surface that lets blame attach; we present the market launch as a single *dependent configuration tuple* whose policy fields are freely chosen above a fixed floor of safety invariants the type discipline discharges uniformly; we specify the maker microstructure as two orthogonal axes (pricing rule × execution discipline) and develop the batch-cleared variant as the form idiomatic to concurrent acceptance over disjoint token pools; and we give the resolution-latch state machine in full, with redeem nested below both consensus finality (the Casper finality fringe, which the runtime already computes for garbage collection) and dispute finality (a window measured in finalised time). Throughout, every required component is shown to map onto apparatus already present in cost-accounted rho plus the combinators; the one genuinely new build is the maker, which is engineering on the substrate rather than an extension of it. The note is written to be implementation-grade: a competent engineer or agent should be able to build from it.

Contents

1 Introduction

A financial contract, in the Peyton Jones–Eber–Seward sense [1], is a thing one *holds*: an agreement to exchange cash flows contingent on time and on observable quantities, generated from a handful of combinators (zero, one, give, and, or, scale, truncate, get). The companion note [2] realises those combinators in the cost-accounted rho calculus [3, 4]—the reflective higher-order process calculus that is the formal core of Rholang [5]—and shows that the realisation turns a description-and-valuation language into an executable settlement language whose executions are resource-safe and atomic by construction. Located authority over funds, atomicity of multi-step settlement, and a linear resource proof discharged before any step fires are the three things the construction supplies that a denotational semantics cannot.

A prediction market is a different kind of object. It is not an instrument but a *venue*: a place where many anonymous participants trade fungible, bearer claims on the outcome of a future event, against one another or against an automated maker, with the claims redeemed for collateral once the event is recognised to have occurred. Kalshi and Polymarket are the two reference points. Kalshi is a regulated, central-limit-order-book exchange whose markets resolve against designated sources under a compliance regime; Polymarket is a permissionless venue whose outcome tokens follow the Gnosis conditional-token pattern, whose liquidity is provided by a constant-product maker and an off-chain order book, and whose markets resolve through UMA's optimistic, bonded oracle. Between them they span the design space we must cover.

The thesis of this note is that the F1R3Fi substrate is unusually well-suited to building such a venue, that most of what a venue needs is *already present* in the cost-accounted calculus, and that the residual—a maker microstructure—is engineering on the substrate rather than an extension

of it. We make the case by inventory and construction. Section 3 separates what a venue needs into four layers and locates each relative to the existing realisation. Section 4 gives the oracle discipline: the trusted computation behind a channel, the impossibility of replicated determinism for genuinely non-deterministic oracles, the blame capability that replaces it, the three strata of oracle specification that decide whether blame can attach, and the reading of the oracle as an *event recogniser* that draws the honest boundary between what the type discipline governs and what it does not. Section 5 presents the market launch as a dependent configuration tuple over a fixed safety floor. Section 6 develops the maker microstructure along two orthogonal axes and gives the batch-cleared join in detail. Section 7 specifies the resolution-latch state machine. Section 8 collects the implementation roadmap: what exists, what is new, and the primitive-by-primitive mapping onto F1R3Fi apparatus.

A note on what is and is not hard. It is worth saying at the outset where the difficulty actually lives, because it is not where a reader coming from the smart-contract world might expect. The *instrument*—a single binary outcome share—is nearly free in the combinator vocabulary: it is $\mathsf{scale}(o, \mathsf{one}(k))$ for an observable o that latches to 1 or 0. The *position layer*—minting, merging and redeeming complete sets of outcome shares—falls out of located purses and the acceptance gate, because its conservation law is a linear funding proof the apparatus already checks. The two genuinely demanding pieces are the *maker* (a contended, stateful, shared object, which is exactly the case the substrate's concurrency story must be steered through with care) and the *oracle* (which is orthogonal to the calculus and is where a prediction market actually lives or dies). The contributions of this note are concentrated accordingly.

2 Background: the apparatus we build on

We recall only what is load-bearing below; [2, 3, 4] give the full account.

The reflective higher-order rho calculus. Names are quoted processes. The grammar is

$$P, Q ::= \mathbf{0} \mid \mathsf{for}(y \leftarrow x)\, P \mid x!(Q) \mid P \mid Q \mid *x, \qquad x, y ::= \ulcorner P \urcorner,$$

with input $\mathsf{for}(y \leftarrow x)P$, output $x!(Q)$, parallel composition $P \mid Q$, the quotation $\ulcorner P \urcorner$ of a process to a name, and the dereference $*x$ recovering the process a name quotes. Reflection—the quote/dereference pair—is what lets contracts, observables, oracles and obligations all be processes, exchanged as names on channels and run by dereference. Parallel composition is associative–commutative: the process soup is an unordered bag, a fact that drives the located-stack discipline below.

Cost accounting as a construction. The cost endofunctor C of [4] adjoins to the host calculus three sorts and a discipline. *Signatures as names:* the name sort is extended so that a signature s is itself a channel that both funds the phlogiston of a communication and authorises the underlying value movement; signatures compose, $s_1 * s_2$ being joint (multi-signature) authority. *Token stacks as first-class terms:* a stack $S ::= () \mid s :: S$ is a temporal monoid whose head is the next token to spend; a balance is a stack. *Wrapping by construction:* every continuation is a signed thunk $\{P\}_s$ that fires only by spending a matching token, so that "no redex fires without being unwrapped" is a sorting invariant preserved by reduction. The single communication rule is replaced by a gated family whose base case is

$$\{\mathsf{for}(y \leftarrow x)\, T \mid x!(U)\}_s \mid s :: S \;\rightsquigarrow\; T\{\ulcorner U \urcorner / y\} \mid S,$$

a communication fires only when a matching token is present, and the consumed token is released as residual.

Three facts we use constantly. (i) **Located purses.** A token stack is held on a channel—$c!(s :: S)$—and the right to spend from it is exactly the right to receive on c; because channels minted by new are unforgeable, authority is possession of the name, never adjacency in the associative–commutative bag [2, §3.2]. (ii) **The acceptance gate.** Multi-step financial atomicity is not a theorem of the rewrite rules but a static analysis at deployment-acceptance: a linear resource proof verifying $\Sigma_s \geq \Delta_s$—supply at least demand—for every signature, before any step fires [2, §3.3]. The operational form is an all-or-nothing reservation from a located slot (the `reserve` contract of [2, Listing 1]). (iii) **Cost is not amount.** Phlogiston counts funded rendezvous, not the magnitude of value moved; scaling a transfer by a large observable still costs one token [2, §3.6].

The combinators as a settlement protocol. A contract is a channel C acquired by

$$C!(s_H,\ s_{Cp},\ h_{Addr},\ c_{Addr},\ \mathit{fund},\ \mathit{res}),$$

with holder and counterparty signatures, the parties' ledger addresses, a located funding slot, and a result channel. An observable is "a persistent channel that returns its current value" [2, §2.2]—a fact we lean on heavily in Section 4, because it makes the oracle channel and the SPJ observable *the same construct.*

3 Gap analysis: four layers

What separates the existing instrument realisation from a Kalshi/Polymarket-class venue decomposes cleanly into four layers. We state each, locate it relative to the existing apparatus, and grade the effort. Table 1 summarises.

The impedance mismatch to resolve first. The combinator contract is *bilateral*: a contract C fixes s_H/s_{Cp} and h_{Addr}/c_{Addr} at acquisition. A prediction-market outcome share is a *fungible bearer position*: owned by whoever bought it, resold to anonymous third parties, redeemed by whoever holds it at resolution. The contract has a counterparty; an outcome token does not. The bridge is the *conditional-token* pattern (Gnosis CTF, Polymarket's actual outcome layer): a *condition* (an oracle plus an outcome-slot count) and three operations—split (lock collateral, mint one share of every outcome onto purses), merge (burn a complete set back to collateral), and redeem (after resolution, exchange the winning shares for collateral). This is the cleanest thing to map onto the substrate, because the conservation invariant $\sum_i(\text{outcome shares}_i) = \text{collateral}$ *is* a located-purse linearity statement, and split/merge are exactly the all-or-nothing reservations the acceptance gate already secures.

Layer 1: position / outcome-token (mostly free)

Required: a complete-set split/merge/redeem primitive over located purses; generalisation from binary to N-way categorical (a mutually-exclusive, collectively-exhaustive partition) and to scalar/range markets (a linear payout across two bracketing outcomes); and a one-shot *resolution latch* distinct from the persistent observable—write-once, with finality, a dispute window, and an invalid-market refund branch. The first two are direct: the conservation law is the funding proof, the partition is a spatial-type fact. The latch is genuinely new state machinery and is treated in full in Section 7. *Effort: low–moderate; congenial to the substrate.*

Layer	What it needs / where it sits	Verdict
Position / outcome-token	CTF split/merge/redeem; conservation = funding proof; N-way and scalar partitions; one-shot latch	Mostly falls out of located purses + gate; latch is new state
Matching / liquidity	CPMM / LMSR / CLOB; cancellation is object-capability; price cell is contended shared state	The large build; batch clearing is idiomatic
Oracle / resolution	Event recognition into consensus; attribution, dispute, finality, invalid handling	Orthogonal to calculus; highest design risk
Microstructure / compliance	Fees, limits, KYC/AML, audit trail	Comparative advantage; native to §6 apparatus

Table 1: The four layers between the instrument realisation and a full venue.

Layer 2: matching / liquidity (the large build)

Required: a continuous double auction (limit/market orders, partial fills, price–time priority, cancellation) and/or an automated market maker (constant-product or logarithmic market-scoring-rule) with its liquidity parameter, maker accounting, and fee/subsidy model. This is the largest gap and the only genuinely hard one. The substrate cuts both ways. Order *cancellation* is idiomatic—a resting order is a funded for-comprehension on a located slot, withdrawable only by the placing signature, so cancellation is object-capability rather than a privileged opcode. RSpace as a concurrent tuplespace is a natural home for the book. But a price level is contended shared state—precisely the case [2, §7] calls hard—so naïve continuous matching reintroduces an ordering/MEV surface. Section 6 develops both continuous and batch-cleared variants and argues the latter is idiomatic to the substrate's "concurrent acceptance over disjoint token pools" discipline. *Effort: high; the centre of the implementation work.*

Layer 3: oracle / resolution (the risk)

Required: a mechanism that brings the recognition of an external, often informally-specified event into consensus, with attribution, dispute, finality, and invalid-market handling. This is where prediction markets live or die, and it is largely orthogonal to the calculus. The capability story helps with *who may resolve* (a resolver signature, or a compliance co-signature); the economic security of resolution is new design. Section 4 gives the discipline in full. *Effort: moderate engineering, high design risk; the part to get right first.*

Layer 4: microstructure / risk / compliance (a comparative advantage)

Required: fee tiers and maker/taker schedules; position and exposure limits; market-maker incentives; and, for a regulated venue, KYC/AML gating and surveillance. Here the substrate is already strong [2, §6]: a native audit trail, compliance co-signatures $\{\mathsf{transfer}\}_{s_{party} * s_{compliance}}$, pre-trade proof obligations, and the spatial types of [2, §8] as the home for position limits (the conservation invariant and "purses do not alias across a market" are exactly the separation predicate Φ_{sep}). *Effort: low–moderate; a reason to favour the regulated path, where this layer is a differentiator against permissionless incumbents.*

One opportunity worth flagging. Because the combinators compose, *conditional* markets ("X resolves YES given Y") and *combinatorial* markets are expressible through and/scale in a way that is

awkward on a flat conditional-token framework. General combinatorial maker pricing is #P-hard, so this is a research lever rather than a v1 feature; we note it and move on.

4 The oracle discipline

The oracle is where the informal world enters the formal substrate. We give the discipline in four moves: the oracle behind a channel; the consensus-determinism problem and its resolution by attribution rather than replication; the oracle specification as the falsification surface; and the oracle read as an event recogniser, which draws the boundary of what the type discipline can and cannot promise.

4.1 The oracle behind a channel

A channel is a quoted process, and nothing in the calculus requires that process to execute inside the shard. We associate a URL of a trusted oracle to a channel x. Any on-shard process may query the oracle by $x!(m)$, where m is a rho-representable instance of a message the oracle answers, and may receive the oracle's information by $\mathsf{for}(msg \leftarrow x)\, P$. These are the same send and receive used for any in-shard rendezvous; the contract author cannot tell, and should not have to, whether the value came from another rho process or from across the network. This is the reflective premise taken seriously: x names a computation that happens to live at a URL.

This makes the SPJ *observable* concrete. The companion note models an observable as a persistent channel returning its current value [2, §2.2]; an oracle channel is precisely that channel. Hence $\mathsf{scale}(o, c)$ with o an oracle channel is already well-typed in the existing vocabulary—the observable was always meant to be this.

The capability story transfers without additional apparatus. Possession of x *is* the authority to query the oracle: a process never handed x cannot name it, cannot send on it, and cannot receive its answer. Oracle access becomes object-capability access control with the same located-authority discipline already applied to funds, now applied to data ingress. A market's resolver capability is "holds the resolution channel"; a compliance-gated resolver is the compound $s_{oracle} * s_{compliance}$.

4.2 Why replicated determinism is the wrong target

A naïve implementation would have every validator dial the URL during reduction. This fails under replicated execution: $x!(m)$ is a live external call—non-deterministic and non-replayable. The oracle may be down, may answer differently between reads, or may answer two validators differently; validators replaying a deployment must see the *same* external value or consensus diverges.

One is tempted to repair this by *signing*: an oracle (or its on-shard agent) submits a signed attestation as a transaction, and the contract's $\mathsf{for}(msg \leftarrow x)P$ reads the posted, consensus-visible datum rather than the live socket. The programming model is unchanged; the value is brought *into* consensus before it is read. For a deterministic oracle—a price feed, a block height—this is exactly right, and the signature makes provenance first-class: in the cost-accounted calculus a signature is a name, so s_{oracle} on the response makes the source auditable by construction.

But for a *genuinely* non-deterministic oracle the repair is illusory, because the non-determinism is at the source and no amount of signing manufactures a reproducibility the oracle never had. The sharp witness is a language model queried at high temperature: re-querying is not verification but a fresh draw. Replay-equality was never the achievable invariant, and insisting on it would rule out exactly the oracles we most want to consult.

The blame capability. The achievable invariant is weaker and is the strongest the non-deterministic case admits: not replicated determinism but *non-repudiable commitment*. A validator signs a block containing an interaction with an oracle. All other validators take this validator at its word until contradicting evidence comes to light; at such time it may be treated as a slashing event (Casper-style) or fed into a reconciliation process. The proposing validator's signature on the block carrying $\mathsf{for}(msg \leftarrow x)P$ *is* the blame capability: $s_{validator}$ names who vouched, the block names what they vouched for, and that binding is unforgeable and permanent. We cannot reproduce the draw, but we can always answer "who committed us to this, and to what"—the question slashing and reconciliation need to fire. This is exactly Casper's accountable-safety shape [6]: equivocation is slashable because the conflicting signatures are evidence; oracle blame is the same move with contradicting evidence sourced externally rather than from a second vote.

4.3 The oracle specification as falsification surface

Attribution alone is hollow without something to be blamed *against*. A bare non-deterministic oracle is unfalsifiable: any answer is consistent with no constraint, so contradicting evidence can never arise, so blame never attaches. The oracle's declared *specification* is what supplies the falsification surface, and it stratifies oracles by how much they constrain. The principle: external rules—distribution over answer sets, admissible answers, source of truth, timeout behaviour—belong in the specification of the oracle, where they become the contract against which later evidence is judged.

Deterministic. A price feed, a block height, the result of an on-chain computation. Admits exact replay; contradiction is any divergence. Blame is sharp.

Distributional. A high-temperature language model *with a declared distribution over answer sets*. No single draw is checkable, but the specification makes aggregate behaviour testable: a validator whose draws depart from the declared distribution over enough samples produces statistical contradicting-evidence, a slashable pattern even though no single answer is "wrong". The specification converts "unrepeatable" into "auditable in aggregate"—exactly enough to keep blame live.

Bare. A non-deterministic oracle with no declared structure. One gets attribution and nothing else. Sometimes that is honestly all the domain affords, and the right move is to be explicit that blame here means "this validator chose the source", not "this validator returned a checkable-wrong value".

Remark 4.1 (The specification is a first-class artifact)**.** *The oracle specification is not prose but a rho-representable term that rides on the oracle channel and is the precise object against which contradicting evidence is later judged. It must be precise enough to falsify against: declared distribution or admissible-answer set, source of truth, and timeout/abstention behaviour. The OSLF-generated logic of [2, §8] is the eventual home for these admissibility predicates, so that "the oracle violated its specification" becomes a checkable assertion rather than a governance argument. The same artifact also declares the dispute-window length (Section 5), so that the latency cost of a market is proportional to the trust assumption it actually makes.*

4.4 The oracle as event recogniser

It would be lovely to have resolution be type-driven, but many events are recognised only informally, or in natural language: "did the ceasefire hold", "is this the most-streamed song of the year". These resolve against a referent that lives in human judgement, not against a checkable predicate, and no

type system adjudicates whether the world made a sentence true. This is not a gap in the calculus; it is the reason the oracle exists. The oracle is the boundary where an informally-recognised event is *coerced* into a rho-representable value—the channel is the seam between the world described in language and a latched value the redeem gate can read. The proposition stays natural language; the *attestation about it* is what becomes typed and attributable.

This draws the honest boundary. What is *typeable*, independent of how informal the underlying event is, is everything downstream of the latch: that the outcomes partition (mutually exclusive, collectively exhaustive); that a complete set sums to collateral; that shares sit on purses that do not alias (Φ_{sep}); that exactly one outcome latches; and that the maker's own purses are resource-sufficient. These are spatial/behavioural facts about the *position*, and they type-check identically whether the resolving oracle is a deterministic exchange feed or a language model reading a news article. What is *not* typeable, and must not be promised as such, is the semantic bridge from world to value—that is the oracle's job, secured by attribution and specification-conformance rather than by typing. The specification's admissibility predicates are the closest the informal case gets to a type; they constrain the attestation's *shape*, never its *correctness against reality*.

Remark 4.2 (Where the natural-language pressure actually lands)**.** *Because informal events are the ones most likely to resolve invalid, ambiguous, or contested, the design pressure from natural-language recognition lands almost entirely on the* resolution path*—the invalid/abstain branch of the latch and the dispute-window length—and leaves the* pricing path *clean. A maker over a clean categorical partition is the easy ninety percent; "invalid ⇒ refund the complete set" must therefore be a first-class settlement path, not an exceptional one. This is why the latch state machine (Section 7) carries the weight it does.*

5 The market-launch configuration tuple

Launching a market is the instantiation of a *dependent record*: some fields' admissible values depend on the values of earlier fields, and the safety floor accumulated below is a *refinement invariant* every instantiation must satisfy. "Launching" is then: instantiate the market-template process at this record; the type discipline discharges the floor uniformly across every cell of the configuration grid. Once the record is fixed and the invariant checks, the rho process is determined—which is the precise sense in which the rest of the implementation is dictated.

$$\begin{aligned}\mathsf{Market} := \langle\ & \underbrace{\mathit{partition},\ \mathit{collateral}}_{\text{instrument}},\ \underbrace{\mathit{pricing},\ \mathit{execution},\ \mathit{economics}}_{\text{microstructure}},\\ & \underbrace{\mathit{oracle},\ \mathit{spec},\ \mathit{dispute},\ \mathit{authority}}_{\text{resolution}},\ \underbrace{\mathit{compliance},\ \mathit{schedule},\ \mathit{limits}}_{\text{access \& policy}}\ \rangle\end{aligned}$$

5.1 The fields

A. Instrument layer (local, free, type-checkable).

partition $\in \{\,\mathsf{Binary},\ \mathsf{Categorical}(n),\ \mathsf{Scalar}(lo, hi, \mathit{brackets})\,\}$. The outcome structure. Binary is $\mathsf{scale}(o, \mathsf{one}(k))$; categorical is an MECE partition into n slots; scalar is a linear payout across two bracketing outcomes. This field alone fixes the *shape* of the conservation invariant ($\sum$ over the slots) and the split / merge / redeem arity. Purely a spatial-type fact.

collateral a located purse, $\mathsf{one}(k)$ for a currency k (REV, a stablecoin, a permissioned USD token). Conservation is "$\sum_i \text{shares}_i$ = collateral in this purse", which is the linear funding proof at the split gate.

B. Microstructure layer (two orthogonal axes plus a dependent third).

pricing $\in \{\,\mathsf{CPMM},\ \mathsf{LMSR},\ \mathsf{CLOB}\,\}$. The maker rule. Purely local to this market's maker purses; one community's LMSR sits beside another's CPMM at zero platform cost.

execution $\in \{\,\mathsf{Continuous},\ \mathsf{Batch}(\mathit{window})\,\}$. Orthogonal to *pricing*—any curve runs under either. This is the one field with a **shard-level externality**: Continuous reintroduces the contended-price-cell / MEV surface and opts into a serialisation regime the shard must support; Batch is clean by construction over disjoint pools. The launcher should *see* this asymmetry; the grid is not a flat menu of equals on this axis.

economics **dependent on** ***pricing***: LMSR demands a subsidiser with bounded loss $b \ln n$ (b a sub-parameter here); CPMM demands liquidity providers earning a fee in basis points; CLOB demands a maker/taker schedule. "LMSR with no subsidy source" is not an incoherent preference but an ill-typed record.

C. Resolution layer.

oracle the resolution channel x. Possession *is* authority; a capability, not a config string.

spec the oracle specification, a first-class rho-representable artifact riding on x, **dependent on its declared stratum** $\in \{\,\mathsf{Deterministic}(\mathit{source}),\ \mathsf{Distributional}(\mathit{answers}, \mathit{dist}),\ \mathsf{Bare}(\mathit{answers})\,\}$. The stratum fixes what counts as contradicting evidence—whether blame can attach—and carries the invalid/ambiguous handling, which for natural-language events is first-class.

dispute the window length, **dependent on** ***spec*****.stratum** and **measured in finalised time** (blocks below the finality fringe since the resolution finalised, so consensus finality is a precondition of the dispute clock even starting). Deterministic feeds want it near zero; distributional / natural-language want a real window. A deterministic feed with a long window is gratuitous latency; a natural-language oracle with a zero window is unsafe—both ill-typed against the stratum.

authority the resolver capability: s_{oracle}, or compound $s_{oracle} * s_{compliance}$ for the regulated shape. The blame capability lives here: $s_{validator}$ on the resolution-bearing block names who vouched.

D. Access & policy layer.

compliance $\in \{\,\mathsf{Open},\ \mathsf{Gated}(s_{compliance})\,\}$. In the gated case every transfer is $\{\mathsf{transfer}\}_{s_{party} * s_{compliance}}$, making the co-signature a structural precondition of settlement. The single field that most distinguishes the Kalshi shape from the Polymarket shape.

schedule trading-open, trading-close ($\mathsf{truncate}\ t$), and resolution-horizon (get). Directly the temporal combinators; no new machinery.

limits optional position/exposure caps: the spatial-type bounds of [2, §8], $\mathsf{K}(\varphi_1, \varphi_2)$-style separation predicates capping how much of a resource a sub-position may claim. Optional in v1; the type-level home is specified.

5.2 The dependent edges and the coherent bundles

The couplings make the launch surface a small set of validated bundles rather than a free twelve-way cross product: *economics* depends on *pricing*; *spec* depends on its stratum; *dispute* depends on *spec*.stratum; *authority* and *compliance* co-vary on the open/regulated fork; and *execution* carries a shard externality the platform has standing in. The honest presentation is therefore a handful of type-checked bundles, e.g.

(LMSR, Batch, subsidised, distributional-oracle, medium window, open)

for a community forecasting market, and

(CLOB, Continuous-or-Batch, maker/taker, designated-resolver, short window, gated)

for a regulated venue, with dependent fields constrained to type-check against their parents.

5.3 The floor: what is *not* in the record

These are fixed across every cell, discharged by the type discipline at launch and by the gates at runtime. The launcher picks policy *above* this line; nobody picks out of the nesting *below* it. That line—configurable policy over fixed safety—is what stops "community selects a point in the grid" from ever meaning "community configures an unsafe market".

1. **Dispute-before-redeem.** No redeem may fire while the dispute window is open.
2. **Redeem below both finalities.** Redeem is nested below *consensus finality* (the resolution block is below the Casper finality fringe—a hard fact, the same boundary the runtime's garbage collection already maintains) *and dispute finality* (the window, in finalised time). The two windows compose; they do not substitute (Section 7).
3. **Conservation of the complete set.** $\sum_i \text{shares}_i$ = collateral, with Φ_{sep} (no aliasing across purses) holding throughout.
4. **One-shot latch.** The resolution is write-once; a second attestation cannot overwrite a latched value.
5. **Acceptance-gate funding proof.** The linear resource proof $\Sigma_s \geq \Delta_s$ before any leg fires, for split, merge, every trade, and redeem.
6. **Invalid ⇒ refund the complete set** as a settled path, not a rescue.

6 The maker microstructure

The maker is the one genuinely new build. It factors into two orthogonal axes: a *pricing rule* (what function turns state into a quote) and an *execution discipline* (when and how trades clear against that quote). We give the pricing rules, then the execution disciplines, then the located-purse realisation of the batch-cleared variant in detail, since it is the one idiomatic to the substrate.

6.1 Pricing rules

Constant-product (CPMM). The maker holds reserves y_i of each outcome's shares and enforces $\prod_i y_i = k$. For a binary market, buying δ YES shares removes them from the YES reserve and requires a deposit that restores the product; the instantaneous price of YES is the normalised reserve

$$p_{\text{YES}} \;=\; \frac{y_{\text{NO}}}{y_{\text{YES}} + y_{\text{NO}}}, \qquad p_{\text{YES}} + p_{\text{NO}} = 1.$$

Simple, the best-understood maker in DeFi, one multiply and one divide per quote; liquidity is thinnest at the extremes (near 0 or 1), where prediction markets often most need it. Liquidity-provider funded.

Logarithmic market-scoring rule (LMSR). Designed for forecasting. The maker tracks outstanding quantities q_i sold and prices through a cost function

$$C(q) \;=\; b \ln \sum_j e^{q_j/b}, \qquad p_i \;=\; \frac{\partial C}{\partial q_i} \;=\; \frac{e^{q_i/b}}{\sum_j e^{q_j/b}},$$

a softmax over outstanding quantities. The payment for a trade $q \to q'$ is $C(q') - C(q)$. Two properties follow: prices are always coherent probabilities ($p_i \in (0,1)$, $\sum_i p_i = 1$ by construction), and the single liquidity parameter b caps the maker's worst-case loss at $b \ln N$ for N outcomes—a known subsidy budget set at launch. Evaluates exponentials and logs; subsidy-funded rather than fee-funded; the canonical multi-outcome prediction maker.

No curve (CLOB). A central limit order book with no maker curve: liquidity is the resting orders themselves, matched by price–time priority. The natural choice for the regulated Kalshi shape.

6.2 Execution disciplines

Continuous. The maker quotes continuously and traders race to hit it. The reserve (CPMM) or outstanding-quantity (LMSR) state is one cell that two traders in the same block contend for. Located purses serialise the *funds* (the maker's inventory sits on unforgeable channels, authority is possession, no ambient draws) but do not by themselves serialise the *price update*: the price-determining state is still one contended cell, and within-block ordering becomes an MEV surface. This is exactly the contended-shared-state case [2, §7] identifies as the hard one. Continuous execution is realisable, but it spends the substrate's concurrency advantage and imports an ordering discipline with externalities.

Batch. Orders arriving within a window are collected, cleared at a single uniform price, and settled together. This fits "concurrent acceptance over disjoint token pools" [2, §7]: orders that do not cross do not contend, the clearing step is one join rather than a race, and uniform pricing removes the within-batch ordering advantage that is the entire MEV prize. A CPMM or LMSR curve can serve as the residual counterparty *inside* the batch, recovering coherent pricing while keeping the concurrency story clean. Batch is the idiomatic cell on this substrate, and we develop it next.

6.3 The order as a funded comprehension; the clearing join

An order is a funded for-comprehension resting on a located book channel. Its fuel and its committed collateral sit on a purse the placing signature owns; cancellation is the right to receive on that purse, available to no one else. Listing 1 shows the order; Listing 2 the uniform-price clearing join.

Listing 1: A resting order: collateral and fuel committed to a located slot; cancellable only by the placing signature.

```
// placeOrder(book, side, limitPrice, qty, sig, fundSlot, ret):
//   reserve qty*limitPrice collateral from the placer's located slot,
//   commit it to a fresh private purse, and rest a cancellable claim
//   on the book channel keyed to this market's batch round.
contract placeOrder(@book, @side, @limit, @qty, @sig, @fundSlot, ret) = {
  new orderId, purse, cancelCh in {
    // acceptance gate: all-or-nothing reservation of committed collateral
    reserve!(fundSlot, sig, qty * limit, *gateRet) |
    for (@(ok, committed) <- gateRet) {
      match ok {
        false => ret!((false, "insufficient funds"))
        true  => {
          purse!(committed) |
          // rest the order on the book for the current batch round
          book!((*orderId, side, limit, qty, sig, *purse, *cancelCh)) |
          // capability to cancel: only the holder of cancelCh may pull it
          ret!((true, *orderId, *cancelCh))
        }
      }
    }
  }
}

// cancel: receiving on cancelCh withdraws the resting order and
// returns the committed collateral to the placer. Object-capability:
// a process never handed cancelCh cannot name it and cannot cancel.
contract cancel(@orderId, cancelCh, @placerSlot) = {
  cancelCh!(Nil) |
  for (@(_, _, _, _, _, @committed, _) <- bookEntry(orderId)) {
    placerSlot!(committed)   // refund to placer's located slot
  }
}
```

Listing 2: Frequent batch auction: collect a round, compute one uniform clearing price, settle all crossing orders against it (with a curve as residual counterparty). Non-crossing orders never contend.

```
// clearBatch(book, round, curve, ret): at the close of a batch window,
//   drain all orders for `round`, compute the uniform price p* at which
//   aggregate demand meets aggregate supply (the curve absorbs the
//   residual imbalance), and settle every crossing order at p*.
contract clearBatch(@book, @round, @curve, ret) = {
  new orders, priceCh in {
    drainRound!(book, round, *orders) |          // gather the round's orders
    for (@os <- orders) {
      // aggregate demand/supply schedules; p* clears the crossing set,
      // curve (CPMM or LMSR) supplies any residual at p* -> bounded subsidy
      uniformPrice!(os, curve, *priceCh) |
      for (@pStar <- priceCh) {
```

```
        // settle: every order with limit on the right side of p* fills,
        // each at the SAME p*; marginal orders fill pro rata.
        // Each fill is one funded transfer -> atomic, one token, located.
        settleAll!(os, pStar) |
        ret!((true, pStar))
      }
    }
  }
}
```

Proposition 6.1 (Batch clearing is MEV-flat within a round)**.** *Under uniform-price batch clearing, every filled order in a round settles at the same p^*, so the value of reordering submissions within the round is zero: no permutation of the round's orders changes any participant's execution price. The only residual ordering value is at the batch boundary (which round an order lands in), which is coarse and publicly timed rather than per-transaction.*

The proposition is the operational content of the claim that batch clearing is idiomatic: the substrate wants disjoint pools cleared by a join, and the join's uniform price is precisely what removes the intra-round ordering prize.

6.4 Maker accounting on located purses

Whichever pricing rule is chosen, the maker's inventory and subsidy sit on located purses, and the conservation law extends to include them. For LMSR the subsidy purse is pre-funded with $b \ln N$ at launch and is the only purse permitted to run a deficit against outstanding shares; for CPMM the reserve purses are funded by liquidity providers whose pro-rata claims are themselves located. The acceptance gate guards every maker interaction exactly as it guards a user transfer: a quote that cannot prove its purse is funded does not fire. *Cost is not amount* applies—a fill is one funded rendezvous regardless of size.

7 The resolution-latch state machine

The latch is the new state machinery the position layer needs. It is distinct from the persistent observable of Section 4.1: an observable may be sampled repeatedly and may return different values over time, whereas a resolution is *write-once with finality*. The latch governs the transition from a tradeable market to a redeemable one, and it is where the two irreversibility notions must be nested in the right order.

7.1 Two finalities, nested

There are two distinct irreversibility guarantees, against two different adversaries. *Consensus finality*: the resolution-bearing block is below the Casper finality fringe; the latched value cannot be rolled back by a reorg. *Dispute finality*: the blame/challenge window has closed; the attributed value can no longer be contradicted by incoming evidence. Redeem must sit downstream of *both*.

Invariant 7.1 (Redeem nesting)**.** *A redeem fires only when the resolution is (i) below the finality fringe* and *(ii) the dispute window measured from that finalised resolution has elapsed. Consensus finality without dispute finality redeems a value that is permanent but possibly wrong (blame can still attach, but the money is gone). Dispute finality without consensus finality closes a challenge window over a block that can still vanish in a reorg, taking the "no disputes arrived" conclusion with it.*

Remark 7.1 (Measure the window in finalised time)**.** *The dispute clock must be keyed to "blocks below the fringe since the resolution finalised", not to proposal height or wall-clock. If the clock started at proposal but the resolution finalised a few blocks deeper, a proposal-keyed window could close before the value is even irreversible—granting dispute finality on a not-yet-consensus-final value. Keying the clock to finalised time makes consensus finality a precondition of the dispute clock even starting, so Invariant 7.1 holds by construction with a single knob.*

Remark 7.2 (The fringe does double duty)**.** *The predicate the redeem gate wants—"is this resolution past finality"—is the same predicate the runtime already computes to decide what state it may garbage-collect. The F1R3FLY implementation keeps only state that can still be reverted, plus the latest finality fringe; so "is this resolution past finality" is definitionally "has this block passed the revertible horizon". The safety predicate for payout and the resource predicate for state-pruning are the same fringe, observed from two sides. The redeem gate reads the liveness boundary the garbage collector already maintains; it asks for no new window into consensus internals.*

7.2 States and transitions

Figure 1 gives the state machine. The states are: OPEN (trading live); PENDING (trading closed at truncate t, awaiting resolution); PROPOSED (an attestation has been posted by s_{oracle} and carried in a block signed by $s_{validator}$, but not yet finalised); FINALISED (the proposal's block is below the fringe—the dispute clock now starts); DISPUTED (a challenge was raised within the window, escalating to reconciliation/slashing); RESOLVED (the dispute window elapsed with the value standing—now and only now is the value latched for payout); INVALID (the market resolved to no admissible outcome, or a dispute upheld invalidity—the refund path); and REDEEMABLE (holders may exchange winning shares, or in the invalid case complete sets, for collateral).

Guards, precisely.

- OPEN→PENDING: block height $\geq$ *schedule*.close (truncate t). New trades refused; existing positions intact.
- PENDING→PROPOSED: a message on the resolution channel x signed by *authority* (s_{oracle}, or $s_{oracle} * s_{compliance}$), carried in a validator-signed block. The attestation must be *admissible* against *spec* (in the answer set; for a categorical market, exactly one slot).
- PROPOSED→FINALISED: the proposing block is below the finality fringe. The dispute clock starts *here*, in finalised time.
- PROPOSED→INVALID: no admissible attestation arrived before *schedule*.horizon (timeout), or the attestation names no admissible outcome.
- FINALISED→DISPUTED: a bonded challenge within the window. For a distributional oracle this includes aggregate-distribution evidence (Section 4.3); the challenge bond funds the reconciliation and is forfeit if the challenge fails.
- DISPUTED→PROPOSED or →INVALID: reconciliation either re-attests a corrected value (and the dispute clock restarts in finalised time) or upholds invalidity (and the offending validator is slashed, Casper-style).
- FINALISED→RESOLVED: the dispute window elapsed with the value standing. *The value is latched here, write-once.*
- RESOLVED→REDEEMABLE and INVALID→REDEEMABLE: the redeem gate opens. By Invariant 7.1 this is downstream of both finalities.

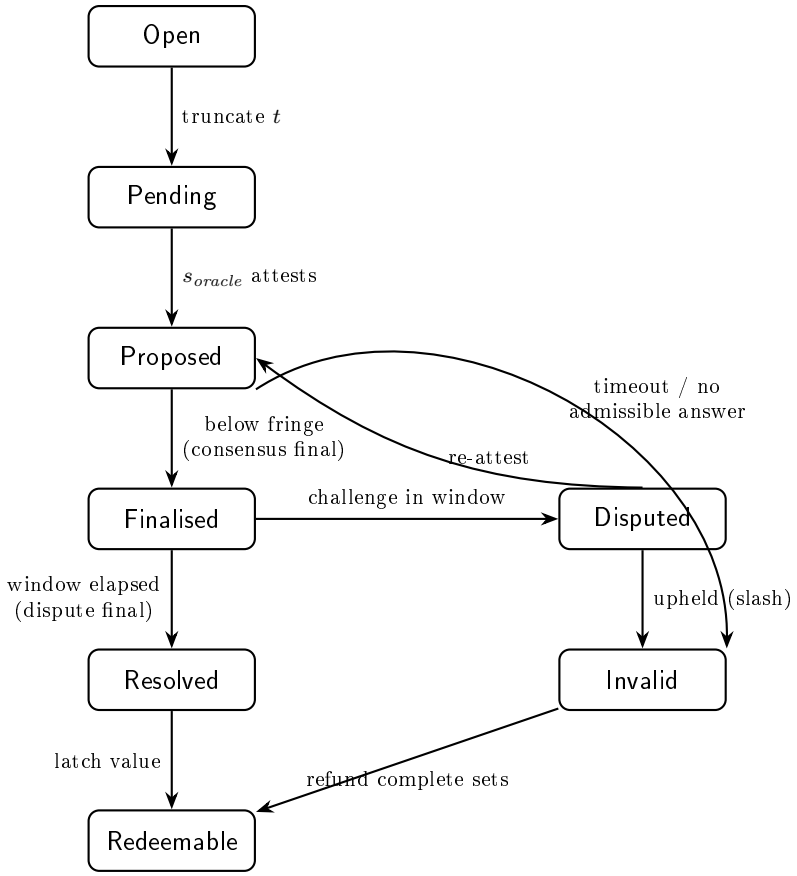

Figure 1: The resolution-latch state machine. Redeem is reachable only through Resolved (a winning-share payout) or Invalid (a complete-set refund), and Resolved is reachable only after both consensus finality (Proposed→Finalised) and dispute finality (Finalised→Resolved).

7.3 The latch in rho

Listing 3 sketches the latch as a one-shot write-once cell whose redeem branch is gated on the conjunction of the two finalities, with the invalid branch refunding complete sets.

Listing 3: The resolution latch: write-once, redeem nested below consensus and dispute finality, invalid refunds complete sets.

```
// latch(x, spec, authority, disputeWin, schedule, marketPurses):
//   x           - resolution channel (the oracle)
//   spec        - admissibility predicate + stratum + window policy
//   authority   - resolver capability (s_oracle [* s_compliance])
//   marketPurses - located purses holding the locked collateral
contract latch(@x, @spec, @authority, @disputeWin, @schedule, @mp) = {
  new resolved, invalidCh in {
    // PENDING -> PROPOSED: accept only an admissible, authorised attestation
    for (@(value, sig, blk) <- x) {
      match (authorised(sig, authority) and admissible(value, spec)) {
        false => invalidCh!("inadmissible or unauthorised")
        true  => {
          // PROPOSED -> FINALISED: wait until blk is below the fringe.
          // finalityFringe is the same boundary the GC maintains.
```

```
          for (_ <- finalityFringe!?(blk)) {
            new clock in {
              // dispute window measured in FINALISED time from here
              startFinalisedClock!(disputeWin, *clock) |
              select {
                // a challenge within the window -> reconciliation
                for (@chal <- disputeCh(blk)) {
                  reconcile!(chal, x, *invalidCh)   // may re-attest or slash
                }
                // window elapsed, value stands -> RESOLVED (latch, once)
                for (_ <- clock) {
                  resolved!(value)                  // write-once cell
                }
              }
            }
          }
        }
      }
    } |
    // REDEEMABLE via RESOLVED: winning shares -> collateral
    for (@v <- resolved) {
      openRedeem!(mp, v)        // pay holders of the winning outcome
    } |
    // REDEEMABLE via INVALID: complete sets -> collateral, first-class path
    for (@reason <- invalidCh) {
      openRefund!(mp, reason)   // every complete set redeems at par
    }
  }
}
```

The `resolved` channel is consumed once; a second attestation finds it empty and cannot overwrite the latched value (the one-shot invariant of Section 5.3). The redeem gate's `finalityFringe` test is read-only access to the consensus liveness boundary; the dispute clock's `startFinalisedClock` keys the window to finalised time, discharging Invariant 7.1 structurally.

8 Implementation roadmap

We collect the build into a primitive-by-primitive mapping (Table 2), a conservation invariant the position layer must maintain, and a phased plan. The organising claim is that no field of the configuration tuple requires a primitive outside cost-accounted rho plus the combinators; the one new build, the maker, is engineering on the substrate.

Invariant 8.1 (Position-layer conservation)**.** *At every reachable state of a market, the collateral locked in the market's collateral purse equals the number of outstanding complete sets, where a complete set is one share of each outcome slot:*

$$\mathit{collateral} \;=\; \sum_{\mathit{minted}} 1 \;-\; \sum_{\mathit{merged}} 1 \;-\; \sum_{\mathit{redeemed}} 1, \qquad \big(*_i\, \mathsf{purse}_i\big) \models \Phi_{\mathrm{sep}}.$$

split increments outstanding sets and locks one unit of collateral; merge decrements and releases one; redeem (post-latch) decrements and releases to the holder of the winning slot (or, in the invalid case, to the holder of any complete set). Each is one acceptance-gate reservation, so the invariant is the gate's linear funding proof and is maintained by construction. Φ_{sep}*—the purses do not alias—is the static no-double-spend.*

Venue component	F1R3Fi apparatus	Status
Outcome share	$\mathsf{scale}(o, \mathsf{one}(k))$, o a latched observable	exists
split / merge	acceptance-gate reservations over located purses; conservation = funding proof	exists (as gate); thin wrapper new
redeem	gated transfer, guarded by the latch	wrapper new
N-way / scalar partition	spatial-type fact; $\sum$ over slots	new (small)
Resolution observable	persistent oracle channel (§4.1)	exists
Resolution latch	one-shot write-once cell (§7)	new
Oracle provenance	s_{oracle} as name; signatures-as-names	exists
Blame capability	$s_{validator}$ on the block; Casper accountable safety	exists (consensus)
Dispute window	finality-fringe read; same boundary as GC	exists (read-only); policy new
Maker (CPMM/LMSR)	inventory on located purses; gate per quote	new (the build)
Order book	resting funded comprehensions on a book channel	new
Order cancellation	object-capability receive on the order purse	exists (idiom)
Batch clearing	join over disjoint pools; uniform price	new
Compliance gate	$\{\mathsf{transfer}\}_{s_{party} * s_{compliance}}$	exists
Audit trail	native ledger of funded rendezvous	exists
Position limits	Φ_{sep}, spatial bounds [2, §8]	type layer (in progress)
Schedule (open/-close/horizon)	truncate / get	exists

Table 2: Primitive-by-primitive map. "exists" means present in the realisation of [2] or the consensus layer; "new" means to be built, with the build's nature noted.

8.1 A phased plan

Phase 0 — position layer. Implement split/merge/redeem over located purses for the binary partition, with Invariant 8.1 as the test oracle. This is the smallest end-to-end slice and exercises the gate, the purses, and the conservation law without any maker or live resolution. Resolve manually (a trusted key) to defer the oracle.

Phase 1 — latch and oracle channel. Implement the latch state machine (Section 7) with a deterministic oracle (a price feed or block height) and a near-zero dispute window, exercising the consensus-finality read against the GC fringe. Add the invalid/timeout branch and the complete-set refund. This makes resolution real on the easy stratum before introducing dispute economics.

Phase 2 — maker, batch-cleared. Implement the order as a funded comprehension (Listing 1) and the batch-clearing join (Listing 2) with a CPMM or LMSR residual. Start batch-cleared because it is the idiomatic, MEV-flat cell; add a continuous mode later only if a specific market demands it, with the externality made explicit. Cancellation is free (the object-capability idiom).

Phase 3 — categorical and scalar partitions. Generalise the position layer and the maker to N-way and scalar markets. The conservation invariant generalises directly; the maker's $\sum_j e^{q_j/b}$ (LMSR) or product (CPMM) ranges over the slots.

Phase 4 — dispute economics and distributional oracles. Add the bonded-challenge game, the reconciliation/slashing path, and distributional-oracle specifications with aggregate-distribution challenges. This is where the genuinely non-deterministic oracles (the high-temperature language model) become usable, and where the design risk is concentrated; it is deliberately last, after the easy strata work end-to-end.

Phase 5 — compliance and limits. Add the $\mathsf{Gated}(s_{compliance})$ configuration and position limits, lifting the regulated-venue bundle. The compliance gate and audit trail already exist; this phase wires them into the market template and the launch tuple.

Configuration surface. Throughout, the market template is parameterised by the tuple of Section 5, with the dependent edges enforced as coherent bundles and the floor of Section 5.3 fixed. A launch is the instantiation of the template at a type-checked record.

9 Conclusion

A financial contract is a thing one holds; a prediction market is a place one trades. The combinators, realised in the cost-accounted rho calculus, already give the first. We have shown that the second is mostly within reach of the same apparatus: the position layer is located purses and the acceptance gate, with conservation as the funding proof; the oracle is a trusted computation behind a channel, with non-repeatable calls secured by validator attribution—a blame capability Casper already supplies—rather than by a replicated determinism that genuinely non-deterministic oracles cannot provide, and with the oracle's declared specification furnishing the falsification surface that lets blame attach; the market launch is a single dependent configuration tuple over a fixed safety floor; the maker is two orthogonal axes whose batch-cleared cell is idiomatic to the substrate's concurrency discipline; and the resolution latch nests redeem below both consensus finality (the same fringe the runtime garbage-collects against) and dispute finality (a window in finalised time). The one genuinely new build is the maker, and it is engineering on the substrate rather than an extension of it. A description language tells you what a contract is worth; a settlement language on this substrate tells you, before a token moves, that it will settle; and a venue on this substrate tells you, before a market opens, the exact discipline under which it will trade and resolve.

Acknowledgements

This note develops work conducted in extended dialogue with Anthropic's Claude, which served as a sounding board throughout—separating the venue layers, pressing on the oracle-determinism question until attribution rather than replication emerged as the right invariant, and drawing out the configuration tuple and the latch nesting. The ideas and any errors are the author's.

References

[1] S. Peyton Jones, J.-M. Eber, J. Seward. Composing contracts: an adventure in financial engineering (functional pearl). ICFP, Montreal, 2000.

[2] L. G. Meredith. Financial Contract Combinators in the Cost-Accounted Rho Calculus. F1R3FLY.io, 2026.

[3] L. G. Meredith. Cost-accounted rho calculus: a spectral decomposition of phlogiston. F1R3FLY.io, 2026.

[4] L. G. Meredith. Continued interactive GSLTs and the cost endofunctor: a construction one level up from cost-accounted Rholang. F1R3FLY.io, 2026.

[5] L. G. Meredith et al. Rholang specification. F1R3FLY.io / RChain Cooperative, 2017–2026.

[6] V. Buterin, V. Griffith. Casper the Friendly Finality Gadget. arXiv:1710.09437, 2017.

[7] R. Hanson. Logarithmic market scoring rules for modular combinatorial information aggregation. Journal of Prediction Markets 1(1):3–15, 2007.

[8] Gnosis. Conditional Tokens Framework. `docs.gnosis.io`, 2019.

[9] UMA. Optimistic Oracle. `docs.uma.xyz`, 2021.

[10] P. Daian et al. Flash Boys 2.0: frontrunning, transaction reordering, and consensus instability in decentralized exchanges. IEEE S&P, 2020.